

Pseudo-SO

João Vítor Maia - 190110007

Arthur Luiz - 232000472

Bruno Henrique - 221022239

¹Dep. Ciência da Computação – Universidade de Brasília (UnB)
CIC0205 - FUNDAMENTOS DE SISTEMAS OPERACIONAIS
29 de junho de 2026

1. Introdução

Nosso pseudo-SO foi implementado em Python, composto por quatro gerenciadores principais: de processos, de memória, de entrada e saída (E/S) e de arquivos. O objetivo é simular, em software, os mecanismos fundamentais de um SO real, com escalonamento de CPU, paginação com substituição LRU, alocação exclusiva de dispositivos e gerenciamento de disco com alocação contígua.

A implementação foi desenvolvida inteiramente com a biblioteca padrão da linguagem, sem dependências externas. O programa é executado via linha de comando da seguinte forma:

```
python dispatcher.py processes.txt files.txt string.txt
```

2. Ferramentas e linguagens utilizadas

- **Linguagem:** Python 3.10+
- **Bibliotecas:** apenas a biblioteca padrão (`collections`, `dataclasses`, `pathlib`, `sys`, `typing`)
- **Testes:** módulo `unittest` da biblioteca padrão
- **Ambiente:** compatível com UNIX; testado em macOS e Linux

3. Componentes do sistema

O sistema é dividido em cinco módulos: `dispatcher.py`, `scheduler.py`, `memory_manager.py`, `resource_manager.py` e `file_system.py`. Um módulo auxiliar `process.py` define a estrutura de dados do processo (PCB).

3.1. Process (PCB)

A classe `Process` é um *dataclass* que representa o Bloco de Controle de Processo. Armazena: PID, tempo de inicialização, prioridade, tempo de CPU, tamanho máximo do conjunto de trabalho (*working set*), requisições de dispositivos (impressora, scanner, modem, SATA), *string* de referência de memória, contador de programa, contador de faltas de página e tempo de espera na fila (usado pelo mecanismo de envelhecimento). O método `is_real_time()` verifica se o processo tem prioridade 0.

3.2. Dispatcher

O `Dispatcher` é o núcleo central do sistema, atua como *bootloader* e CPU. Suas responsabilidades são:

- **Fase de boot** (`load_inputs`): lê e interpreta os três arquivos de entrada, cria os PCBs e inicializa o sistema de arquivos com os segmentos pré-ocupados do disco.
- **Ciclo de máquina** (`run`): implementa o laço principal de simulação de relógio (*clock*). A cada *tick* de 1 ms, o dispatcher: (1) admite processos cujo tempo de inicialização chegou; (2) solicita ao escalonador o próximo processo; (3) tenta alocar os recursos de E/S necessários, se indisponíveis, reencaminha o processo à mesma fila sem rebaixar prioridade; (4) acessa a página de memória correspondente à instrução atual; (5) decrementa o tempo de CPU e aplica preempção ou finalização; (6) dispara o envelhecimento das filas de usuário.
- **Relatório pós-execução**: após todos os processos terminarem, executa as operações de arquivo pendentes e imprime o mapa de ocupação do disco e o número de faltas de página por processo.

3.3. Scheduler

O Scheduler gerencia quatro filas de processos prontos conforme ilustrado na Figura 1 da especificação:

- **Fila de tempo real** (prioridade 0): política FIFO estrita, sem preempção por quantum. O processo ocupa a CPU até encerrar.
- **Filas de usuário** (prioridades 1, 2 e 3): sistema de múltiplas filas com realimentação (*multilevel feedback queue*). Quantum de 1 ms. Ao esgotar o quantum, o processo é rebaixado para a fila de prioridade inferior (`preempt`). Processos bloqueados por indisponibilidade de recursos retornam à **mesma** fila sem penalidade (`requeue`).

Para prevenir *starvation*, o método `update_wait_times_and_age` é invocado a cada *tick*: processos nas filas 2 e 3 que aguardam por 10 ou mais *ticks* consecutivos são promovidos à fila imediatamente superior (técnica de *aging*). O escalonador suporta até 1000 processos simultâneos e mantém a contagem global atualizada ao admitir (`admit`) e ao concluir (`complete`) processos.

3.4. Memory Manager

O MemoryManager simula a memória principal com 20 *frames* de 1 KB cada, particionados em:

- **8 frames** exclusivos para processos de tempo real.
- **12 frames** exclusivos para processos de usuário.

Ao ser admitido, cada processo recebe uma pré-carga de uma página (`preload`), evitando falta de página na primeira instrução. A cada instrução, `access_page` verifica se a página referenciada já está carregada (*page hit*) ou precisa ser trazida do disco (*page fault*). Em caso de falta, aplica-se o algoritmo de substituição **LRU no escopo local**: a página menos recentemente usada **do próprio processo** é substituída quando o *working set* ou a partição física está cheia. Faltas de página não consomem tempo de CPU do processo. Ao encerrar, `free_process` devolve todos os *frames* ao sistema.

3.5. Resource Manager

O `ResourceManager` controla a alocação exclusiva dos dispositivos de E/S disponíveis: 1 scanner, 2 impressoras, 1 modem e 2 dispositivos SATA.

A alocação segue a política **tudo ou nada**: o processo obtém acesso somente se todos os recursos que solicitou estiverem disponíveis simultaneamente (`request`). Não há preempção de dispositivos, se os recursos estiverem ocupados, o processo aguarda na fila sem rebaixamento de prioridade. Processos de tempo real ignoram completamente o gerenciador de recursos, pois a especificação determina que eles não necessitam de dispositivos de E/S. Ao finalizar, `release` devolve exatamente as quantidades alocadas ao pool global.

3.6. File System

O `FileSystem` simula o armazenamento secundário como um vetor de blocos, onde blocos livres são marcados com 0 e blocos ocupados armazenam o nome do arquivo.

- **Criação** (`create`): utiliza o algoritmo *first-fit*, varrendo o disco a partir do bloco 0 em busca da primeira lacuna contígua suficiente. Retorna (`True`, `start_block`) em caso de sucesso ou (`False`, `-1`) se não houver espaço.
- **Deleção** (`delete`): processos de tempo real podem deletar qualquer arquivo; processos de usuário só deletam arquivos de sua propriedade. Retorna (`True`, `''`) em caso de sucesso, (`False`, `'not_found'`) ou (`False`, `'permission'`) em falha.
- **Persistência**: arquivos criados permanecem no disco mesmo após o encerramento do processo criador.
- **Relatório**: `print_disk_map` exibe o mapa de ocupação do disco ao final da execução.

4. Sincronização e compartilhamento de recursos

A especificação menciona o uso de semáforos, monitores ou troca de mensagens para sincronização, *quando se fizer necessário*. Na abordagem adotada, o pseudo-SO é implementado como um programa **single-threaded**: o `Dispatcher` é o único fio de execução real, e todos os gerenciadores são acessados sequencialmente a cada *tick* do relógio simulado.

Dessa forma, não existe concorrência real entre os “processos” simulados, eles são objetos Python gerenciados de forma determinística pelo laço do dispatcher. A exclusão mútua sobre os recursos de E/S é garantida **por lógica**, pelo `ResourceManager`, sem necessidade de primitivas de sincronização do sistema operacional. Mecanismos como semáforos ou monitores seriam necessários apenas se cada processo simulado fosse implementado como uma *thread* ou processo do SO host, o que não é o caso.

5. Conclusão

O pseudo-SO implementado cobre os quatro gerenciadores exigidos pela especificação, com escalonamento por múltiplas filas com realimentação e *aging*, gerência de memória por paginação com LRU local, controle exclusivo de dispositivos de E/S e sistema de arquivos com alocação contígua *first-fit*. A escolha pela abordagem single-threaded eliminou a necessidade de sincronização explícita, simplificando a implementação sem comprometer a correção do comportamento simulado.